

UNIVERSIDAD POLITÉCNICA DE PACHUCA

Ingeniería en Tecnologías de la Información e Inovación Digital

DESARROLLO E IMPLEMENTACIÓN DE COLUMNAS Y LAYOUTS EN JETPACK COMPOSE

Curso: Desarrollo de Aplicaciones Móviles

Autor: Hernández Ramírez Viky Naomi

Profesor: Montoya Cerro Miguel Angel

Lugar y Fecha: Zempoala, Hidalgo, 05 de octubre de 2026

Índice

1. Introducción **5**

1.1. Contextualización 5

1.2. Planteamiento del Problema 5

1.3. Objetivos 5

1.3.1. Objetivo General 5

1.3.2. Objetivos Específicos 5

1.4. Justificación 6

1.5. Estructura del Documento 6

2. Marco Teórico **6**

2.1. Definición de Conceptos Clave 6

2.1.1. Column 6

2.1.2. Modifier 6

2.1.3. Modificadores de Dimensión 6

2.1.4. Formato de Imagen WebP 7

2.1.5. LazyColumn 7

2.2. Teorías y Antecedentes 7

3. Desarrollo **7**

3.1. Análisis del Cambio de Paradigma: Column vs. LinearLayout 7

3.2. Guía Paso a Paso y Desarrollo de la Práctica 8

3.2.1. Paso 1: Selección de la Plantilla Base 8

3.2.2. Paso 2: Configuración del Proyecto 8

3.2.3. Paso 3: Estructura Inicial de MainActivity 9

3.2.4. Paso 4: Importaciones Básicas de Compose 10

3.2.5. Paso 5: Definición de la Función Composable con Column 11

3.2.6. Paso 6: Importaciones Detalladas de Compose 12
3.2.7. Paso 7: Verificación en el Emulador con Fondo Rojo 13
3.2.8. Paso 8: Importación de Recursos Gráficos 13
3.2.9. Paso 9: Aplicación Completa en Ejecución 14
3.2.10. Paso 10: Implementación del Bloque item en LazyColumn 14
3.3. Ejemplo de Código Implementado 15
3.4. Explicación Narrativa del Comportamiento Visual y Estructura 16

4. Conclusiones **16**
4.1. Aportes y Limitaciones 16
4.2. Recomendaciones y Proyecciones 17

5. Anexos **17**
5.1. Anexo A: Referencia del Material de Apoyo 17
5.2. Anexo B: Código Completo de la Práctica 17

6. Referencias **17**

Índice de figuras

1. Selección de la plantilla *Empty Activity* en Android Studio 8
2. Configuración de parámetros del proyecto 9
3. Estructura inicial de la clase `MainActivity` 10
4. Importaciones básicas de Jetpack Compose 11
5. Función composable `app()` con `Column` y tres `Text` 12
6. Importaciones detalladas de Compose 12
7. Resultado de la `Column` con fondo de color en el emulador 13
8. Importaciones para cargar imágenes con `painterResource` 13
9. Aplicación completa en ejecución con imagen y textos 14
10. Bloque `item` con `Image` dentro de `LazyColumn` 15

Índice de tablas

1. Contraste entre `Column` en Compose y `LinearLayout` vertical 7

1. Introducción

1.1 Contextualización

Como desarrolladora de aplicaciones Android, he notado que migrar al enfoque declarativo de Jetpack Compose requiere comprender bien cómo se organizan los elementos en pantalla. Este reporte describe el funcionamiento de la maquetación vertical con `Column` y con `LazyColumn`, su versión de carga perezosa.

1.2 Planteamiento del Problema

Al probar Jetpack Compose descubrí que, si se colocan varios componentes sin un contenedor de diseño, todos se dibujan encimados en la esquina superior del lienzo. Para lograr una interfaz ordenada fue necesario estudiar el uso de columnas, el encadenamiento de modificadores de tamaño, color y alineación, y el manejo del desplazamiento cuando el dispositivo cambia de orientación.

1.3 Objetivos

1.3.1 Objetivo General

Desarrollar una aplicación móvil con interfaces verticales basadas en el contenedor `Column`, aplicando correctamente los modificadores, empleando imágenes optimizadas y resolviendo el desbordamiento de contenido mediante `LazyColumn`.

1.3.2 Objetivos Específicos

- Crear un proyecto en Android Studio con la plantilla *Empty Compose Activity*.
- Mostrar cómo se superponen los componentes de texto cuando no existe un layout que los organice.
- Aplicar el componente `Column` y analizar el encadenamiento de modificadores.
- Agregar imágenes en formato WebP con descripciones de contenido para lectores de pantalla.
- Detectar las limitaciones de `Column` al girar la pantalla y corregir el desbordamiento utilizando `LazyColumn`.

1.4 Justificación

Esta práctica me ayuda a consolidar los fundamentos de la maquetación declarativa en Android, al distinguir entre los contenedores estáticos y aquellos que permiten desplazamiento y mayor dinamismo en la interfaz.

1.5 Estructura del Documento

El presente documento se organiza en cuatro secciones principales: primero el marco teórico, donde se definen los conceptos clave; posteriormente el desarrollo de la práctica, que incluye el análisis del cambio de paradigma, la guía paso a paso y la explicación narrativa del comportamiento visual; finalmente, las conclusiones, anexos y referencias en formato APA 7.

2. Marco Teórico

2.1 Definición de Conceptos Clave

2.1.1 Column

Es el contenedor básico de Jetpack Compose que coloca a sus hijos uno debajo de otro, de arriba hacia abajo. Funciona de forma similar a un `LinearLayout` vertical del sistema clásico de vistas XML.

2.1.2 Modifier

Es el objeto que permite modificar la apariencia y el comportamiento de un componente: su tamaño, color, espaciado, alineación e interacciones dentro de la interfaz declarativa.

2.1.3 Modificadores de Dimensión

Son los modificadores que determinan si un elemento ocupa toda la pantalla o solamente el ancho disponible de su contenedor padre, como `fillMaxSize` y `fillMaxWidth`.

2.1.4 Formato de Imagen WebP

Es un formato de compresión de imágenes muy usado en desarrollo móvil, ya que disminuye el peso de los archivos sin afectar de manera notable la calidad visual ni el consumo de memoria.

2.1.5 LazyColumn

Es un contenedor de desplazamiento vertical muy eficiente. Equivale al `RecyclerView` del sistema clásico, con la diferencia de que únicamente compone y dibuja los elementos que se encuentran visibles en pantalla.

2.2 Teorías y Antecedentes

Al preparar esta práctica estudié cómo Jetpack Compose aprovecha mejor los recursos del dispositivo mediante composables perezosos. También comprendí que la interfaz se vuelve a dibujar solo cuando cambia el estado de la aplicación, lo que evita trabajo innecesario sobre la jerarquía de vistas.

3. Desarrollo

3.1 Análisis del Cambio de Paradigma: Column vs. LinearLayout

A continuación presento una comparación entre el método tradicional con archivos de diseño XML y el enfoque declarativo que apliqué en esta práctica:

Criterio	Layout Clásico (XML)	Jetpack Compose
Componente Equivalente	`LinearLayout` vertical	Composable `Column`
Definición de Estilos	Atributos XML o archivos de estilos	Encadenamiento sobre `Modifier`
Renderizado de Listas	`RecyclerView` + `Adapter`	`LazyColumn` directo
Invocación en Código	Inflado de vistas / *View Binding*	Llamada directa a composables

Tabla 1: Contraste entre `Column` en Compose y `LinearLayout` vertical

3.2 Guía Paso a Paso y Desarrollo de la Práctica

3.2.1 Paso 1: Selección de la Plantilla Base

Abrí Android Studio y seleccioné la opción de crear un nuevo proyecto. En la captura se observa el asistente *New Project*, donde elegí la categoría *Phone and large screens* y la plantilla *Empty Activity*, la cual genera automáticamente un proyecto con Jetpack Compose habilitado y la estructura mínima para comenzar a desarrollar.

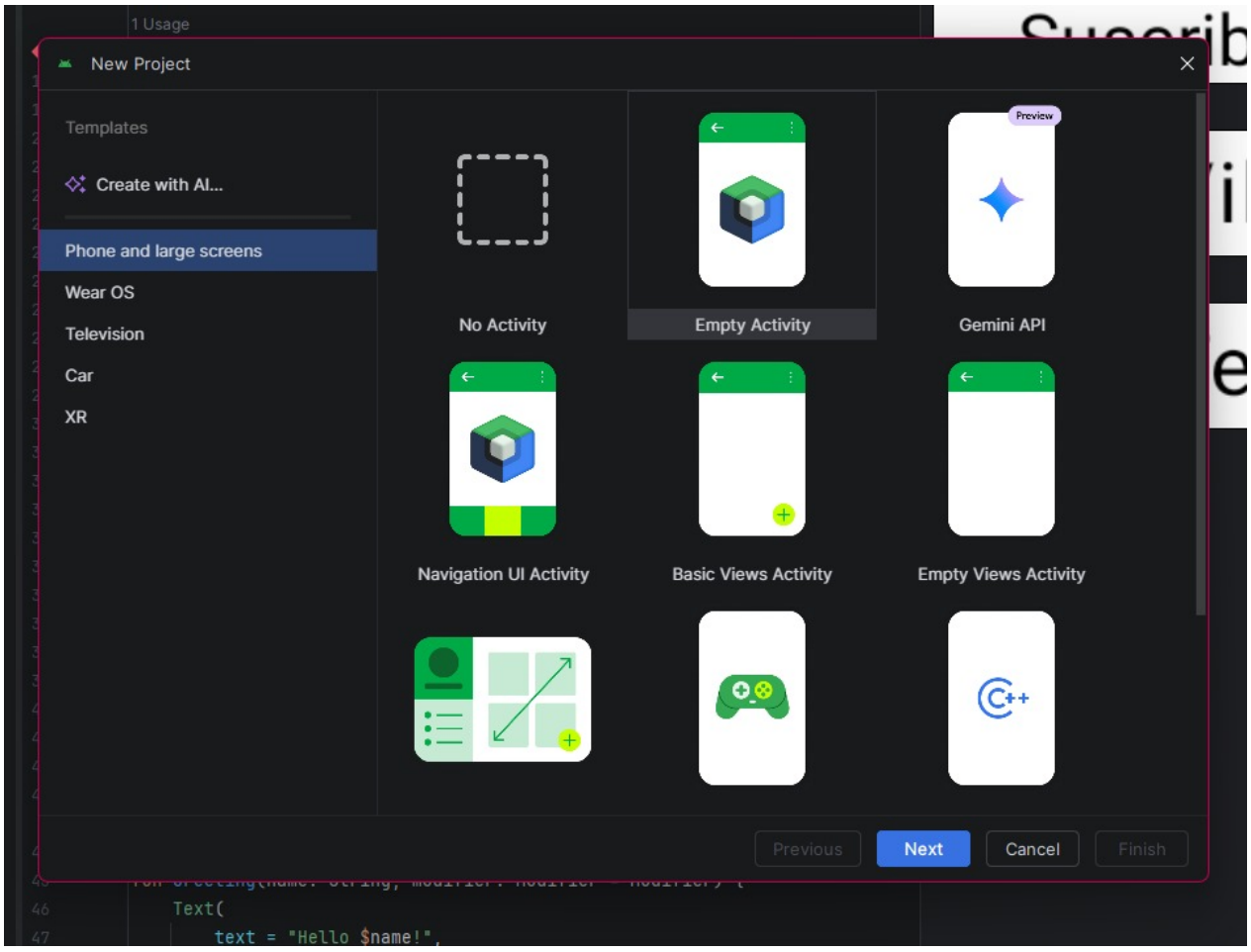


Figura 1: Selección de la plantilla *Empty Activity* en Android Studio

3.2.2 Paso 2: Configuración del Proyecto

Configuré los parámetros del nuevo proyecto. En la captura se observa el formulario con el nombre "Column My Application", el paquete `com.example.columnmyapplication`, la ubicación de guardado, el SDK mínimo (API 24, Android 7.0) y el lenguaje de configuración de compilación *Kotlin DSL*.

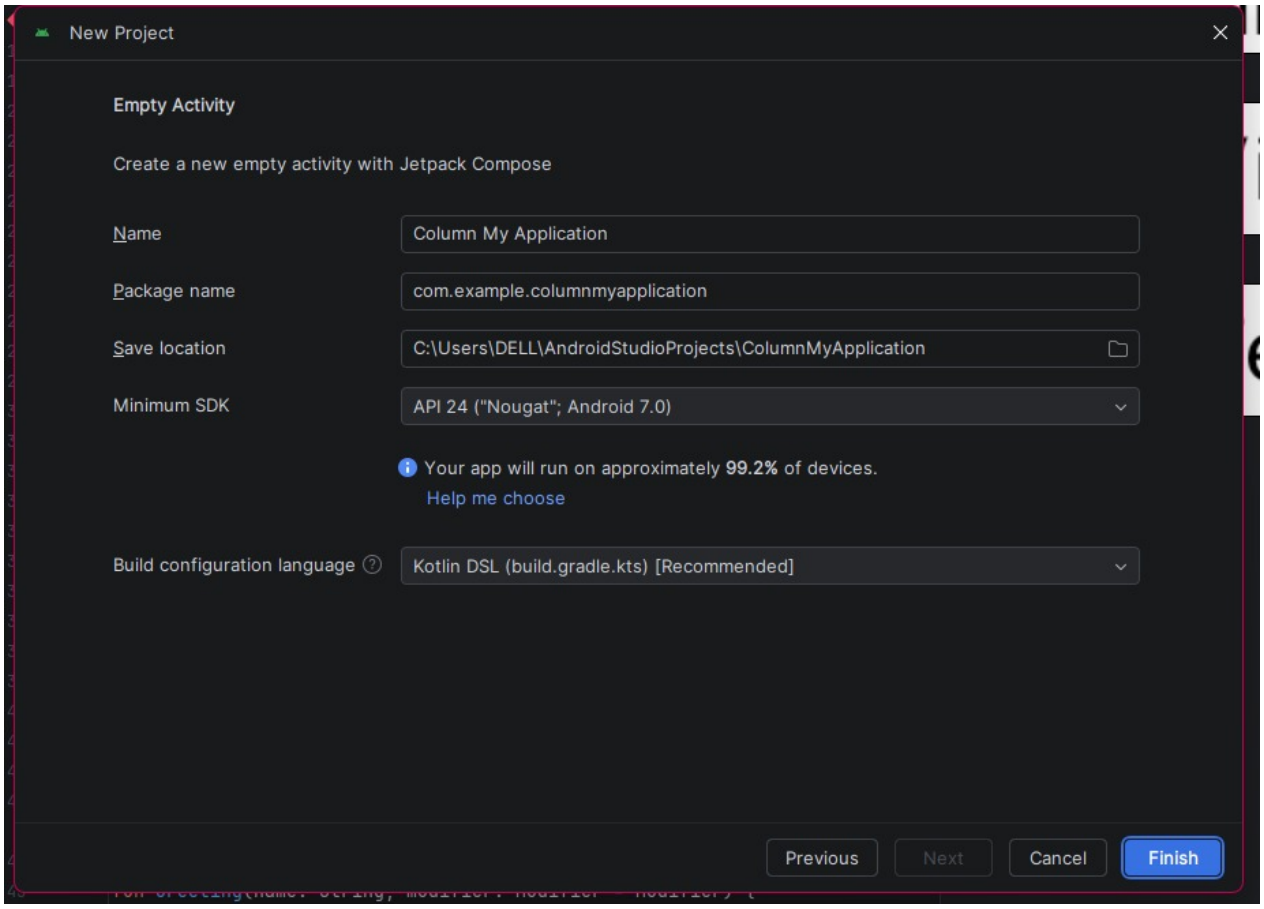


Figura 2: Configuración de parámetros del proyecto

3.2.3 Paso 3: Estructura Inicial de MainActivity

Una vez creado el proyecto, revisé la estructura inicial de la clase `MainActivity`. En la captura se observa la clase heredando de `ComponentActivity`, con el método `onCreate` que llama a `enableEdgeToEdge()` y a `setContent`, el cual por ahora está vacío.

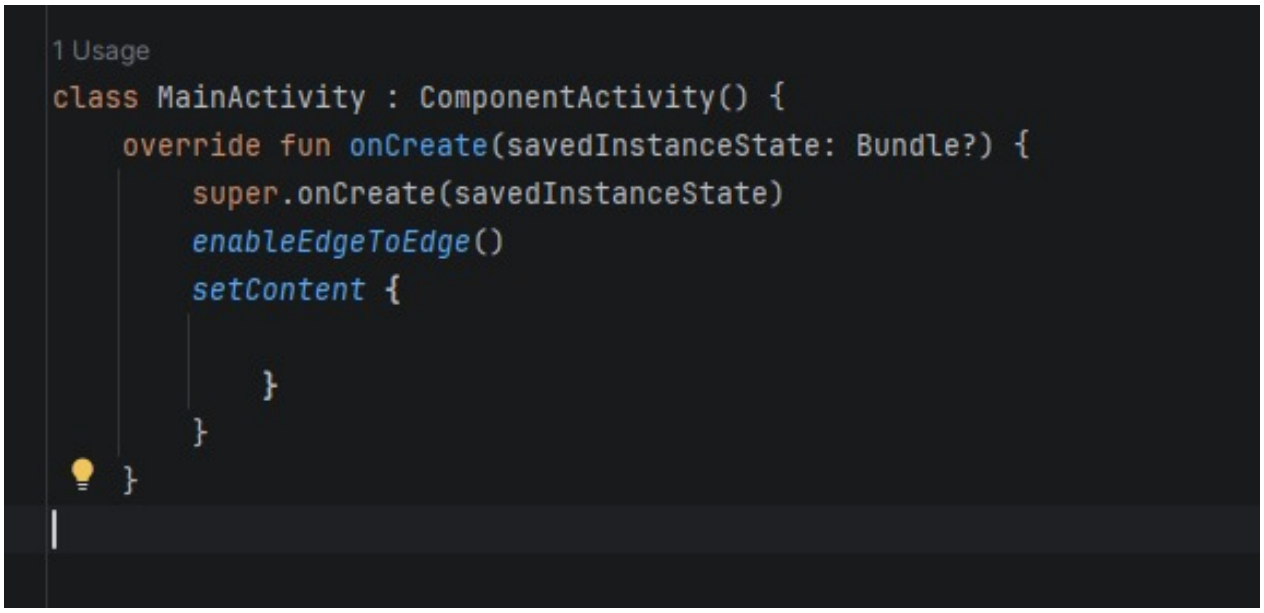


Figura 3: Estructura inicial de la clase `MainActivity`

3.2.4 Paso 4: Importaciones Básicas de Compose

Agregué las importaciones necesarias para trabajar con los composables de Jetpack Compose. En la captura se observa la sección de `import` que incluye `Composable`, `Modifier`, `Color`, `Preview` y `Column`.



Figura 4: Importaciones básicas de Jetpack Compose

3.2.5 Paso 5: Definición de la Función Composable con Column

Escribí la función composable `app()` con la anotación `@Preview` para poder visualizarla en el editor. En el código se observa el contenedor `Column` con el modificador `fillMaxSize()` y tres composables `Text` que se apilan verticalmente.

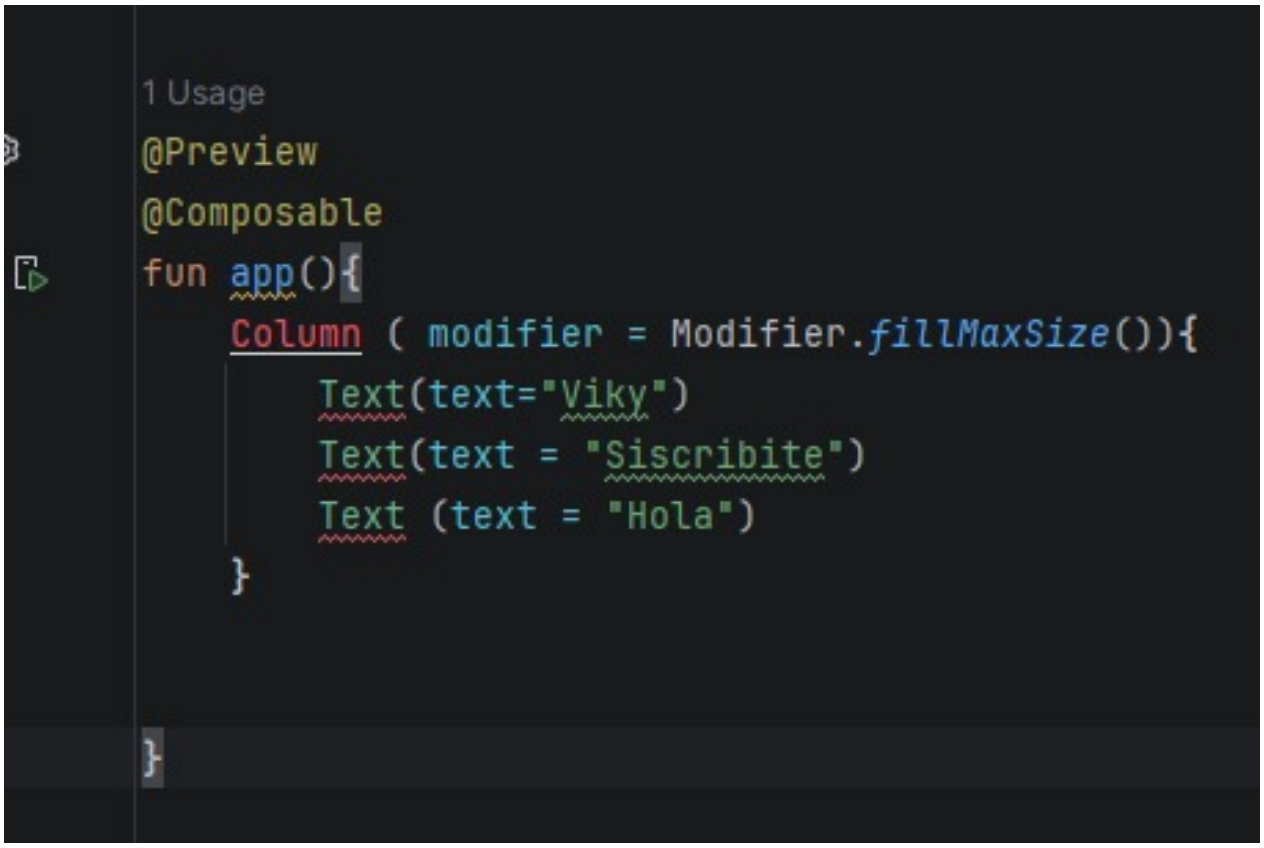


Figura 5: Función composable `app()` con `Column` y tres `Text`

3.2.6 Paso 6: Importaciones Detalladas de Compose

Añadí las importaciones específicas que requiere el proyecto para funcionar correctamente. En la captura se observan los paquetes `androidx.compose.runtime`, `androidx.compose.ui`, `androidx.compose.f` y el tema personalizado de la aplicación.

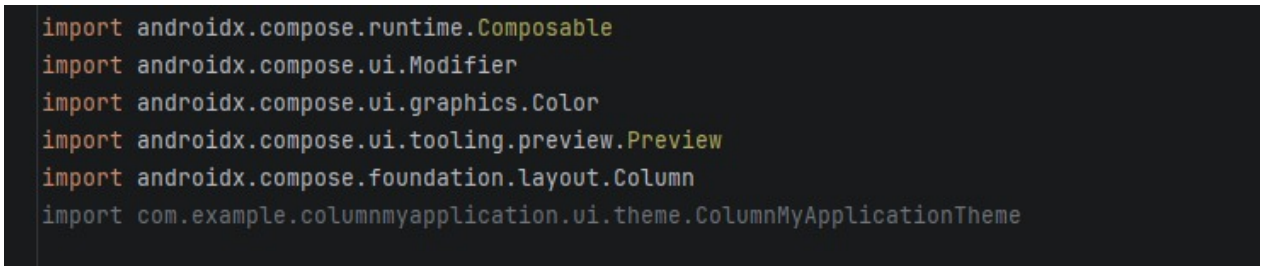


Figura 6: Importaciones detalladas de Compose

3.2.7 Paso 7: Verificación en el Emulador con Fondo Rojo

Ejecuté la aplicación en el emulador y comprobé el resultado visual. En la captura se observa el fondo rojo completo que cubre toda la pantalla, con los textos “Viky”, “Suscribete” y “Hola” apilados verticalmente.

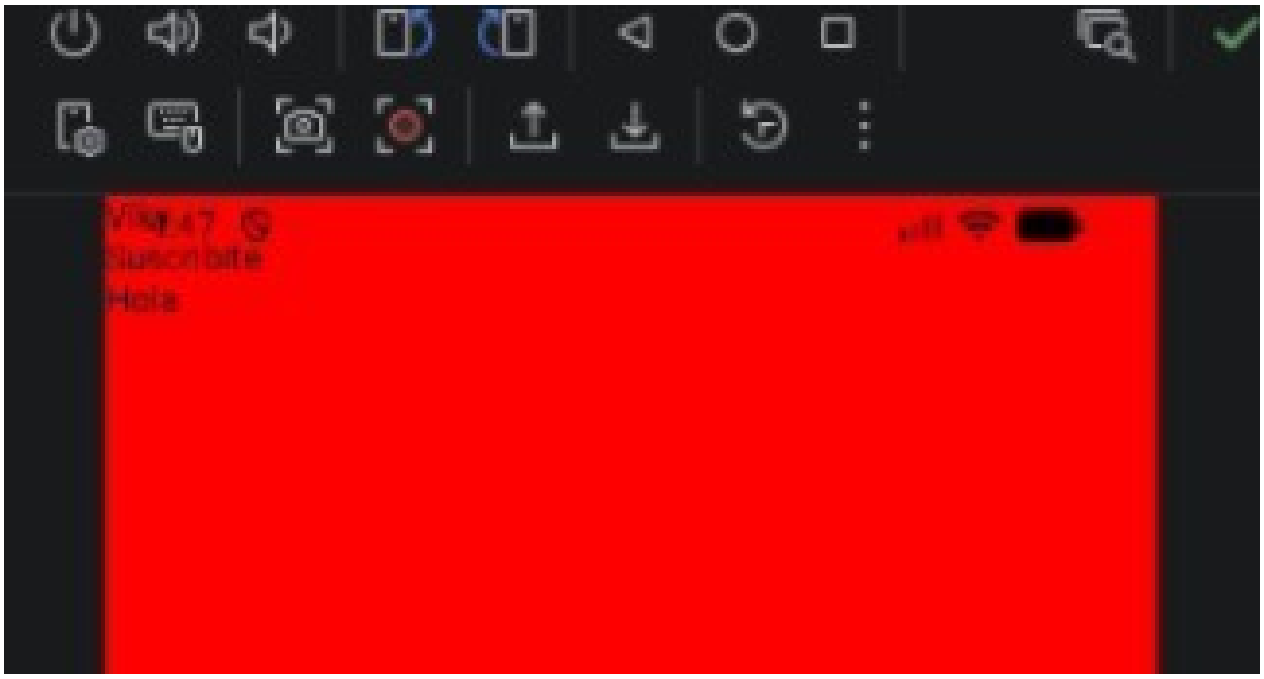


Figura 7: Resultado de la `Column` con fondo de color en el emulador

3.2.8 Paso 8: Importación de Recursos Gráficos

Incorporé las importaciones necesarias para usar imágenes dentro de los composables. En la captura se observan los `import` de `Column`, `Image`, `painterResource` y `Preview`.

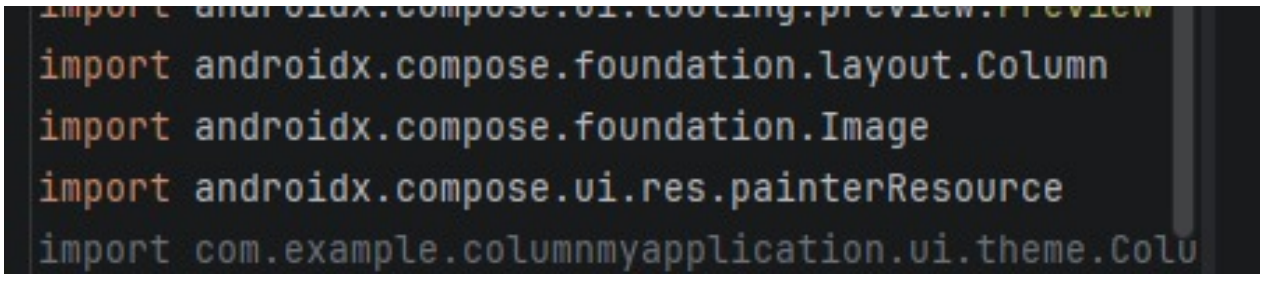


Figura 8: Importaciones para cargar imágenes con `painterResource`

3.2.9 Paso 9: Aplicación Completa en Ejecución

Ejecuté la aplicación completa y verifiqué el resultado final. En la captura se observa el código de `MainActivity` con el tema y el `Scaffold`, y a la derecha el emulador mostrando la imagen WebP del personaje junto con los textos sobre el fondo rojo.



Figura 9: Aplicación completa en ejecución con imagen y textos

3.2.10 Paso 10: Implementación del Bloque item en LazyColumn

Reemplacé la `Column` por un `LazyColumn` y agrupé los componentes dentro de un bloque `item`. En el código se observa el composable `Image` con `painterResource`, `contentDescription` y los modificadores `fillMaxWidth()` y `height(300.dp)`.



Figura 10: Bloque `item` con `Image` dentro de `LazyColumn`

3.3 Ejemplo de Código Implementado

A continuación se muestra el bloque principal de la función composable utilizada, seguido de su descripción:

```
@Preview
@Composable
fun app() {
    Column(modifier = Modifier.fillMaxSize()) {
        Text(text = "Viky")
        Text(text = "Suscribete")
        Text(text = "Hola")
    }
}
```

Listing 1: Función composable principal con Column

Descripción: Este bloque define una función composable llamada `app()` que utiliza el contenedor `Column` con el modificador `fillMaxSize()` para ocupar toda la pantalla. Dentro se declaran tres componentes `Text` que se apilan verticalmente.

```
item {
    Image(
        painter = painterResource(id = R.drawable.hola),
        contentDescription = "Imagen principal",
        modifier = Modifier
```

```
6                 .fillMaxWidth()
7                 .height(300.dp)
8         )
9         Text(text = "Viky")
10    }
```

Listing 2: Bloque item con Image dentro de LazyColumn

Descripción: Este bloque muestra la inclusión de una imagen dentro de un `item` de `LazyColumn`, aplicando `fillMaxWidth()` y una altura fija de 300 dp. El parámetro `contentDescription` garantiza la accesibilidad para lectores de pantalla.

3.4 Explicación Narrativa del Comportamiento Visual y Estructura

Durante la práctica observé que el orden de los modificadores influye en el resultado visual. Al colocar primero el tamaño y después el color, el fondo cubrió toda la superficie de la columna y no solo el área que ocupaba el texto.

También comprobé que `Column` es adecuada cuando hay pocos elementos y su cantidad es fija. Sin embargo, cuando el espacio vertical disminuye, como al pasar a modo horizontal, no ofrece desplazamiento por sí sola y los componentes inferiores quedan cortados.

Al pasar a `LazyColumn` el problema desapareció. Este componente solo crea las vistas que están visibles, lo que reduce el uso de memoria y permite un desplazamiento fluido en cualquier orientación.

4. Conclusiones

4.1 Aportes y Limitaciones

- Concluyo que el enfoque declarativo de Jetpack Compose facilita la creación de interfaces verticales, ya que no es necesario escribir archivos XML por separado.
- Aprendí que el orden en que encadeno los modificadores cambia directamente la posición y el aspecto de un componente en la pantalla.
- Verifiqué lo importante que es considerar la rotación del dispositivo, y que `LazyColumn` es la solución adecuada para evitar que el contenido se desborde.

- Confirmé que usar formatos modernos como WebP ayuda a optimizar recursos y a mantener un buen desempeño en las aplicaciones móviles.

Como limitación, el proyecto se probó únicamente en un emulador *Medium Phone API 35*.

4.2 Recomendaciones y Proyecciones

Se recomienda continuar con el estudio de otros contenedores como `Row`, `Box` y `LazyRow`, así como profundizar en el manejo de estado y en la composición de layouts anidados.

5. Anexos

5.1 Anexo A: Referencia del Material de Apoyo

Para realizar y comprobar esta práctica me apoyé en el siguiente material de consulta sobre desarrollo Android:

- **Referencia del recurso:** Android Developers Video Tutorials — *Layouts and Modifiers in Jetpack Compose*.
- **Enlace de acceso:** `https://developer.android.com/jetpack/compose/layouts`

5.2 Anexo B: Código Completo de la Práctica

El código completo del proyecto `ColumnaCompose`, así como las prácticas 1 y 2, se encuentra disponible en el siguiente repositorio de GitHub:

- **Repositorio:** *Desarrollo de Aplicaciones Móviles Septiembre-Diciembre 2026*
- **Enlace de acceso:** `https://github.com/sicxvkyn-glitch/Desarollo-de-Aplicaciones-M-viles`

6. Referencias

Android Developers. (2024). *Jetpack Compose Layouts*. Google. `https://developer.android.com/jetpack/compose/layouts`

Android Developers. (2024). *Lists and Performance in Compose*. Google. https://developer.android.com/jetpack/compose/lists

Google Developers. (2023). *WebP Image Format Documentation*. https://developers.google.com/speed/webp

Hernández Ramírez, V. N. (2026). *Desarrollo de Aplicaciones Móviles Septiembre-Diciembre 2026* [Repositorio de GitHub]. GitHub. https://github.com/sicxvkyn-glitch/Desarollo-de-Apl